预习 4.1~4.2

4.1 语法制导的定义

- 4.1.1 语法制导定义的形式
  - 1、属性分类
    - **综合属性**：若 b 是 A 的属性，且其依赖的 $c_1, c_2, \cdots, c_k$ 是产生式右部符号的属性或 A 的其他属性 。
      - 特性：一般通过分析树中该结点的子结点属性值来计算 。
    - **继承属性**：若 b 是产生式右部某个文法符号 X 的属性，且其依赖的属性是 A 的属性或右部文法符号的属性 。
      - 特性：一般由结点的兄弟结点、父结点和自己的属性值来计算 。
  - 2、关键特性
    - 同一个文法符号的综合属性集和继承属性集的交集应为空 。
    - 终结符只有综合属性，其值由词法分析器提供（即记号的属性），因此没有计算终结符的语义规则 。
  - 3、副作用
    - 实际应用中，一些语义规则的目的不是计算值，而是产生打印值、输出中间代码、修改全程量等操作（如过程调用或程序段） 。
    - 可以把它们看作定义了左部非终结符的一个虚拟综合属性 。
  - 4、属性文法：语义规则函数无副作用的语法制导定义 。
- 4.1.2 综合属性
  - 1、S 属性定义：仅仅使用综合属性的语法制导定义 。
  - 2、计算特性：分析树各结点属性的计算可以自下而上地完成（从叶结点到根结点）
  - 3、核心概念：
    - 注释分析树：每个结点的属性值都标注出来的分析树 。
    - 分析树的注释/修饰：计算各结点属性值的过程 。
- 4.1.3 继承属性
  - 1、应用场景：编程语言的一些构造的属性依赖于它们所在的上下文 。
  - 2、典型示例：将类型信息传递给一张声明表中的各个标识符（如将 int 类型沿分析树向下传递给各个 id 结点，存入符号表） 。
  - 3、重写性质：虽然重写 SDD 使之仅使用综合属性总是可能的，但有时会使文法失去简洁和直观，不如使用继承属性自然 。
- 4.1.4 属性依赖图
  - 1、定义：描绘分析树结点属性之间互相依赖关系的有向图 。
  - 2、构造方法：
    - 分析树上每个结点的每个属性在依赖图上都有一个结点 。
    - 如果属性 b 依赖于属性 c，则从 c 的结点到 b 的结点画一条有向边 。
    - 为过程调用（有副作用的规则）引入虚拟综合属性结点 。
- 4.1.5 属性计算次序
  - 1、拓扑排序：有向无环图结点的一种排序，使得有向边只会从排序中先出现的结点指向后出现的结点 。
  - 2、概念上的实现步骤（分析树方法）：
    - 根据基础文法构造输入的分析树 。
    - 构造属性依赖图 。
    - 对依赖图的结点进行拓扑排序，得到语义规则的计算次序 。
    - 按该次序计算属性，得到输入串的翻译 。
  - 3、分析树方法的缺点：编译速度很慢，因为属性计算次序是在编译过程中动态确定的 。如果依赖图有环，该方法会失败 。
  - 4、改进方法（为了提高编译速度，在构造编译器时确定次序）： ⑦

4.2 S属性的自下而上计算

> 掌握**S 属性定义、语法树、综合属性自下而上计算**的核心思想，理解 LR 栈如何承载语义信息，为后续学习 L 属性、自上而下翻译打下基础。

- 4.2.1 语法树
  - 1、语法树（Syntax Tree）核心特点
    > 分析树的浓缩、简化表示、中间表示。
    - **结构简化**：算符、关键字作为分支结点（内部结点），而非叶结点；单非终结符链可省略，结构更紧凑。
    - **结点含义**：内部结点代表运算，子结点为运算对象；叶结点为标识符、数字等基本运算对象。
    - **作用价值**：作为**中间表示形式**，可实现 "先分析生成语法树，再基于语法树翻译"，也可辅助边分析边翻译；C、Java 编译器常显式构造语法树。
    - **属性关联**：可像分析树一样，将属性附加到语法树结点，支撑语法制导翻译。
- 4.2.2 构造语法树的语法制导定义
  - 1、语法树结点数据结构
    > 用记录实现，分两类结点：
    - **算符结点**：含算符标记、左子树指针、右子树指针；
    - **基本运算对象结点**：含对象类型标记、值 / 符号表指针。
  - 2、核心构造函数（返回新建结点指针）
    - `mkLeaf(id, entry)` ：创建标识符叶结点，`entry` 为符号表条目指针；
    - `mkLeaf(num, val)` ：创建数字叶结点，`val` 为数字值；
    - `mkNode(op, left, right)` ：创建算符分支结点，`left/right` 为左右子树指针。
  - 3、表达式语法树的 S 属性定义
    > 为含+、*的表达式文法添加语义规则，定义综合属性 nptr（存储语法树结点指针），核心规则如下：
    - E→E+T： `E.nptr = mkNode('+', E₁.nptr, T.nptr)`
    - E→T： `E.nptr = T.nptr`
    - T→TF： `T.nptr = mkNode('*', T₁.nptr, F.nptr)`
    - T→F： `T.nptr = F.nptr`
    - F→(E)： `F.nptr = E.nptr`
    - F→id： `F.nptr = mkLeaf(id, id.entry)`
    - F→num： `F.nptr = mkLeaf(num, num.val)`
- 4.2.3 S 属性的自下而上计算
  - 1、S 属性定义核心
    - 仅含综合属性（属性值由子结点属性计算得出），无继承属性，是最简单的语法制导定义。
  - 2、自下而上计算原理
    > 依托LR 分析器实现，核心是扩展分析栈：
    - **栈结构**：每个栈元素含state（状态 / 文法符号）和val（综合属性域）；
    - **计算时机**：归约前，用栈顶右部符号的属性，计算左部符号的综合属性；
    - **归约逻辑**：产生式 A→XYZ 归约时，从栈顶取Z.z、Y.y、X.x，计算A.a并存入原X的栈位置，调整栈顶指针。
  - 3、工具关联
    - 该方法是Yacc等编译器生成工具的核心实现逻辑，理解其原理可支撑后续工具使用。
- 难点总结
  - 区分**分析树**与**语法树**的差异，理解语法树的简化逻辑；
  - 理解综合属性nptr的含义（语法树结点指针）及语义规则的执行逻辑；
  - 掌握 LR 分析栈的扩展结构，以及归约过程中**属性值的栈位置映射**；
  - 理解事件驱动式语义规则与传统程序流程的区别。

by 幕布用户0301723 由 幕布 发布